Optimization of Infiniband for Scientific Applications

Gregory Johnson[†], Darren J. Kerbyson, Mike Lang
Performance and Architecture Lab (PAL), Los Alamos National Laboratory, USA
{djk,mlang}@lanl.gov

Abstract

The achievable performance on Infiniband networks is governed by the latencies and bandwidths of communication channels as well as by contention within the network. Currently Infiniband statically routes messages and thus do not take into account dynamic loading of the channels. By interrogating the network routing tables we quantify the contention that occurs for a number of communication patterns using a large-scale (1024 processor) system. Empirical data confirms our contention calculation almost exactly. Custom routing tables are defined that provide both optimum and worst-case performance for a large-range of communication patterns. Performance differences can be as large as 12x (from optimum to worst-case). Two large-scale applications show a run-time improvement of between 10-20% and up to 40% improvement in just their communication time when using optimized routing tables. The approach taken is applicable to many Infiniband systems, and we expect the performance improvements to be even greater on larger-scale systems.

1. Introduction

The performance of an HPC system is determined by the capabilities of the compute nodes, the capabilities of the interconnection network, and the way in which applications exercise these resources. Infiniband can be classed as a commodity-based interconnect that is increasingly used in HPC systems. Though not originally developed for the HPC market, Infiniband networks are found in other areas including telecommunications and storage. The specification of a standard for Infiniband has encouraged many manufacturers to deliver both HCAs (Host Channel Adapters), and switch components. Installed large-scale HPC systems include the Thunderbird cluster at Sandia National Laboratory [15] and System-X at Virginia Tech [16]. The performance of an individual communication channel boasts high bandwidth with current networks achieving a 4x DDR (Double Data Rate) performance of ~1.6GB/s, and single 0-byte message latency from HCA to HCA of 1.3µs [114]. Infiniband switch chips currently contain a 24-port cross-bar which allows 12-ary fat-trees, as well as other topologies, to be built. The main manufacturer of Infiniband technology, Mellanox, produces cross-bars chips that are packaged into switch chassis containing a total of 288 ports supplied by multiple vendors, such as Voltaire and Silver Storm.

In this work we undertake a performance analysis and optimization of a system that has a single 288-port Infiniband switch. We are not concerned with the peak performance of individual components of the network but rather examine how it performs for particular communication patterns that arise in many large-scale scientific applications. We consider a communication pattern to consist of many communications between distinct processor pairs in the system. The majority of parallel scientific applications correspond to the SPMD (Single Program Multiple Data) programming model in which all application processors perform a computation step at approximately the same time followed by a communication step at the same time. By considering the communication patterns we are effectively examining and optimizing the performance of the communication steps in an application.

The motivation for this work comes from our application performance experience with adaptive and static routed interconnects. Current Infiniband networks use static routing to route messages between source and destination nodes. That is, a deterministic path is always used to transfer messages between one particular node and another. The routing does not adapt with the dynamic loading on the network and contention in particular communication channels can result. Similar routing is used in Myrinet networks. Adaptive routing uses the dynamic loading in the network to determine how each message segment should progress. For instance in the Quadrics QSNet fat-tree networks [4] message segments are adaptively

[†] Currently at Google (gwj@google.com}

978-1-4244-1694-3/08/$25.00 ©2008 IEEE

routed when progressing up the tree and statically routed when progressing down the tree (to eliminate dead-lock). The added capabilities within the Quadrics network can alleviate much of the contention issues that arise in statically routed networks and hence provide higher levels of achievable performance for a given channel bandwidth. Adaptive routing is not available on current Infiniband networks.

A further challenge is that compute-nodes within a system are not always configured such that their logical order, as visible to an application, matches the physical ordering on the interconnect switches. If logical neighbors are not physical neighbors, as far as possible, on the switch chips much of the traffic for local neighbor communication will be routed through switches in upper fat-tree levels, thus increasing contention. In other networks the problem is alleviated by excess bandwidth. In the Cray XT3, the communication channel bandwidth exceeds that that can be injected by a single node and thus reduces contention [6]. Another challenge that can intensify contention is the increasing number of cores per socket, with multi-core processors even more MPI processes could share the interconnect network.

In Section 2 static routing as used within a typical Infiniband network is described along with details on the experimental system used in this work. The characteristics of communication patterns are described in Section 3 along with a model to calculate the contention that arises for a particular pattern given a set of routing tables. In section 4 we analyze the contention that arises in our experimental system and show the result of using optimized routing tables for a range of communication patterns. Empirical data is used to confirm the contention in Section 5. The impact of this optimization on the performance of two applications is quantified in Section 6. Conclusions from this work are discussed in Section 7.

1.1. Contributions of this work

We demonstrate that, for a particular application communication pattern, the performance of an Infiniband network is greatly affected by contention within the network. Contention is quantified for a communication pattern by interrogating the routing tables contained within each switch chip in the network. The current emphasis with Infiniband routing is on the algorithms used by the subnet manager to quickly discover the nodes within the network fabric, but little is done after the fabric is operational.

The methods we use to optimize the network performance arise from an application centric viewpoint. Rather than optimizing particular network features such as bandwidth, latency or collectives, we consider the application requirements and customize the fabric accordingly. Optimization was achieved by a strict control of the routing tables to match typically occurring communication patterns. The approach can be extended to include more of a dynamic application-aware routing process. A single optimized routing table is presented that encompasses a range of communication patterns that commonly occur in scientific codes. We envision that such an optimization could be utilized in a large number of systems given the increasing popularity of Infiniband. Work on incorporating optimized routing tables in OpenSM, similar to those presented here, was reported in [17]. Recent novel adaptive routing methods for Fat-trees include [5].

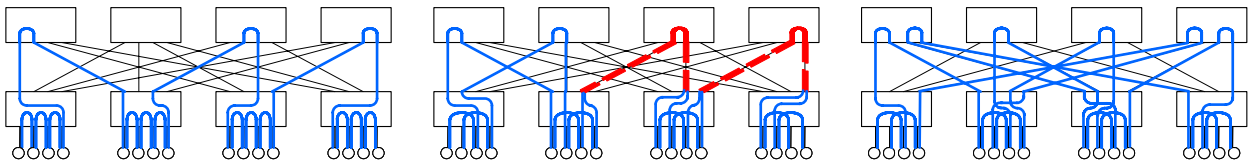
The optimization of a 1024-processor system results with much improved communication times on individual communication patterns, as well as 10-20% improvement in run-time on two large-scale applications. The actual improvement in just the communication component of the application runtime is also examined and shows a 20-40% improvement. Also since contention increases with scale, and cluster sizes are increasing, the results of this work will be more relevant in the future.

2. Infiniband networks for high performance computing

Infiniband is based on an extensive specification [7]. It was designed for enterprise network replacement rather than specifically for HPC. The switching hardware lacks some features that exist in other interconnects, such as hardware collectives and adaptive routing (e.g. Quadrics QsNet [4] and the Blue Gene/L network [1]). Even with these limitations Infiniband fabrics have scaled to thousands of nodes. Currently 24-port switches enable 12-ary fat-tree networks as well as other reduced connectivity networks to be implemented. A switch chassis, consisting of a total of 288-ports, is often the network building block in large-scale Infiniband systems.

Routing in Infiniband networks is destination-based static routing. When a packet arrives on any cross-bar input port, the output port is determined by looking up the destination LID in the LFT which has one entry per LID in the network. The routing table (or map) for the entire fabric consists of all of the LFTs. In normal operation the SM discovers the network and builds the routing table using one of several methods, for example: minimum hop or UP/DOWN.

Static routing of messages can result in contention in the network when many messages are sent between distinct pairs of nodes at the same time. For instance consider a small 16-port network shown in Figure 1. A



(a) N_i -> N_i±1 **(b) N_i -> N_i±2 (with contention)** **(c) N_i -> N_i±2 (no contention)**

Figure 1. Example routing on a 16 node statically routed quaternary fat-tree network

2-level quaternary tree is used for illustration purposes. A set of communications is shown by the blue curves in Figure 1(a) – in which each node N_i communicates with $N_i \pm 1$ (nodes are numbered sequentially from left to right). In Figure 1(a) all communications proceed at the same time as no contention occurs – only one message progresses upwards from any first level switch.

A further example set of communications is shown in Figure 1(b) and 1(c) in which each node N_i communicates with node $N_i \pm 2$. The possible contention in the network occurs when two messages, originating from the same first level switch, are routed over the same up-link. This is shown in the example of Figure 1(b) by the red dashed lines. However, it is possible for all of these communications to progress without contention if the routing table in each switch permits – an example of routing the same set of messages free of contention is shown in Figure 1(c).

The definition of routing tables that provide contention free communications for particular communication patterns is the subject of this work and described further in Section 4. It builds upon knowledge of commonly occurring communication patterns in scientific codes as described in Section 3.

It should be noted that the Infiniband standard also provides for Multi-path routing (MPR). Multiple identifiers can be assigned to each network node, and different routing paths between each distinct pair of identifiers can exist. MPR can provide a means of alleviating contention but not eliminating it. As an extension of this work, MPR could also be used to switch between multiple communication patterns that occur during application execution.

3. Application communication patterns

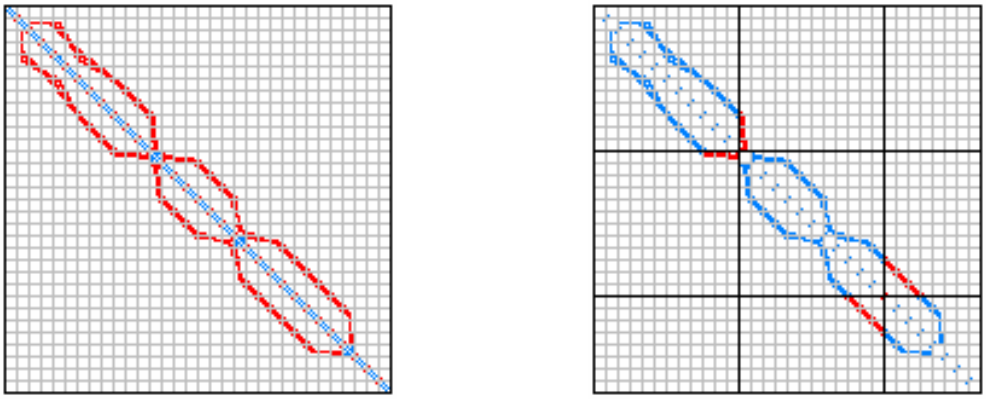
Scientific applications commonly contain a limited number of communication activities such as boundary exchanges between logically adjacent processors, and collective operations. Most conform to the SPMD (Single-Program-Multiple-Data) style – that is each processor performs the same operation but on a different sub-grid of the overall global problem. When a communication activity occurs all processors typically perform it at the same time. The nature of the operation is the same for all processors. For boundary exchanges, the communication takes place between many processor-pairs

From recent analysis of scientific applications [3,8], the number of communication patterns is usually limited and the majority of applications exhibit regular communication patterns. Even when irregular patterns occur, they often contain regular patterns and can be mostly represented by a set of communication shifts [10]. Additionally the communicating processing-pairs, and hence the communication pattern, may be dynamic – it may change from one application iteration to the next but can persist for a reasonable amount of time before changing [3].

3.1. Physical communication topology

The layout of application tasks onto processors in a system and the way in which messages are routed determines the physical communication topology. Communications that occur between processors within the same compute-node need to be distinguished from those that occur between processors on different nodes. Intra-node communications typically use the nodes internal network, such as HyperTransport or shared memory, and the external HPC network is used for inter-node communications. As an example an irregular communication pattern is shown in Figure 2(a). The vertical axis represents the sources of the communication (from 0 to $P-1$), and the horizontal axis represents the destination (again from 0 to $P-1$). Intra-node communications are shown in blue and inter-node communications are shown in red. In this example, each node contains 4 processors and the extent of intra-node communications are shown by the faint horizontal and vertical lines.

The communications occurring within the first level of Infiniband switches is shown in Figure 2(b) for the example communication pattern. In a similar way to the intra-node example, the intra-switch communications are shown in blue, and the communications requiring the next level of switches are shown in red. The extent of each switch (48 PEs = 12 nodes × 4 PEs per node) is shown by the black horizontal and vertical lines. The communications that occur between nodes on the same switch can be handled simultaneously for different communicating nodes. Note that the bottleneck in this case is simply



(a) 4-way nodes (blue = intra-node, red = inter-node) **(b) first level of 12-port switches (blue = intra-switch, red = inter-switch)**

Figure 2. Example 128 processor communication pattern

the number of messages originating from the same node since they share a single network NIC.

An important aspect of this work is in the efficient utilization of the routes between any two level-1 switches using switches in level-2. Even though in Figure 2(b) it appears as though there are only a few messages between the 1st and 2nd switch (as well as between the 2nd and 3rd switch), if these are routed via the same switch in an upper level of the network, they will effectively be serialized and share the same route down to the destination node. However, given the fat-tree nature of the Infiniband network, multiple routes are available but, due to the static routing employed in each of the network switches, they are not necessarily employed for a given communication pattern.

3.2. Communication routing and contention

For a particular communication pattern the physical channels used within the network can be determined by analyzing the LFTs (routing tables) within each of the switches in the network. Since the routing is static, we can precisely determine the path taken by each message and hence determine the maximum contention that will occur for a particular pattern. The maximum contention will determine the performance of a particular communication stage within an application. For simplicity we have assumed that all messages within a communication pattern are of the same size in the following analysis. This assumption can be relaxed if necessary.

We usea routing array, $\mathbf{RA}^m$, in a similar way to [13] for each of the messages, m, contained within a communication pattern:

$$\mathbf{RA}^m = \begin{bmatrix} P_1^{1,1}, P_2^{1,1} \ldots P_{24}^{1,1} \| & \ldots & \| P_1^{1,n}, P_2^{1,n} \ldots P_{24}^{1,n} \\ \vdots & & \vdots \\ P_1^{L,1}, P_2^{L,1} \ldots P_{24}^{L,1} \| & \ldots & \| P_1^{L,n}, P_2^{L,n} \ldots P_{24}^{L,n} \end{bmatrix} \quad (1)$$

Each element of $\mathbf{RA}^m$, $P_x^{z,y}$, represents a port x on switch y at communication level z in the network. Individual switches are separated by the double vertical lines in (1). The number of communication levels in this description is twice the number of switch levels – one for each communication direction (both up to and down from a switch level). For instance the communication links from the compute nodes to the first level switches are represented by $z=1$, and from the first level switches to the compute nodes are $z=2$. For a fully-populated 288-port switch with 288 compute-nodes, the $\mathbf{RA}^m$ has 288 columns and 4 rows. $P_x^{z,y}$ is a binary number and is set to 1 if message, m, uses the particular channel in the network.

The contention array (**CA**) contains the number of messages that use each of the communication channels in the network for a particular communication pattern, and is the sum of the **RA**s for all messages, N_{msg}, within a communication pattern:

$$\mathbf{CA} = \sum_{i=1}^{N_{msg}} \mathbf{RA}^i \quad (2)$$

The maximum contention, *MaxC*, for a particular communication pattern is simply the largest value within **CA**. *MaxC* determines the time for a particular communication pattern in an SPMD application and can be thought of as a multiplicative factor on the messaging time for large messages. *Max*C can be minimized by optimizing the routing of all messages within a communication pattern.

Note that the calculation of the maximum contention is just one use of the **CA**. For instance the number of unused links, the distribution of contention, and the use of MPR can also be examined.

4. Infiniband routing table analysis

The routing tables within the Infiniband fabric can be stored to file when using a modified OpenSM. In addition, this modified version also enables a predefined map (set of routing tables) for the whole network to be loaded at any point. Part of a routing table is shown below in which the local identifier (LID) specifies a destination address in the system, the output port is specified by a value between 1 and 24, and the Destination Info contains global identifiers (GUIDs) for the destination device. Switch output ports in the range 1 thru 12 are up-links in the network (to a switch in a higher level in the network), and ports in the range 13…24 are down-links which on the lowest switch level are connected to the compute nodes. LIDs in the range 5…10 for a single switch are shown below. Note that devices in the network also include the switches themselves (e.g. LID 8 below).

```
LID      Out    Destination
         Port   Info
0x0005   004  : (Adapter portguid 0x0008f1040396fa01: 'MT25208')
0x0006   005  : (Adapter portguid 0x0008f1040396fb01: 'MT25208')
0x0007   006  : (Adapter portguid 0x0008f1040396fc01: 'MT25208')
0x0008   010  : (Switch portguid 0x0008f10400400905: 'ISR9288')
0x0009   008  : (Adapter portguid 0x0008f1040396f405: 'MT25208')
0x000a   009  : (Adapter portguid 0x0008f1040396f505: 'MT25208')
...
```

A typical routing map was obtained using the modified OpenSM's dump facility. In addition two custom routing maps were defined and are described in Section 4.1 below which provide an optimized set of routing tables as well as a worst-case set of routing tables for a 2-level, 288-port, Infiniband cluster. The contention that arises from these maps, as well as from the typical map, is compared in Section 4.2 for a number of communication patterns.

4.1. Routing table definition

In the following we consider one particular communication pattern – that of a logical shift: P_i -> $P_i \pm s$ where s is the shift distance and ranges from 1 to 288. As described in Section 3 this particular communication pattern can be considered a superset of other patterns including those occurring from 1-D, 2-D and 3-D regular data decomposition schemes. Three separate network maps are used which we refer to as *Typical*, *Worst-case*, and *Optimized*:

Typical – a map built by the OpenSM subnet manager after a reboot, and that represents a typical case from an analysis of many such maps for the network when using the logical shift pattern.

Worst-case – a map in which a routing table for each lowest level switch was manually defined that routes messages destined for all compute-nodes attached to any other switch through a single up-link. The maximum contention in this case is equal to the number of compute-processors attached to a switch (48 in our test system).

Optimized – a map in which a routing table for each lowest level switch was manually defined that allows 12 messages (one per up-link) to be routed simultaneously to any other lowest level switch. Thus this routing table best enables a logical shift communication pattern of any shift distance to be embedded in the network with no contention.

For a two level network, as shown in Figure 1, and considering the level-1 switches to be number L1 thru L24, the worst-case routing tables are defined by:

For each Level 1 switch n = 1 .. 24
    For all Compute Nodes not connected to Ln
        use output port n/2

and Optimized routing tables are defined by:

For each Level 1 switch n = 1..24
    For all Compute Nodes not connected to Ln
        use output port (Compute Node) % 12

As a system scales in size, the performance of a typical map will approach that of the worst-case. For all three maps the routing down-wards in the network is deterministic as there is at most one route that can be taken. No alteration was made to the routing tables in these cases. An optimized map for networks with a greater number of switch levels could be similarly defined in which each higher level of switches spreads out the communication traffic for the shift communication pattern across their up-links.

5. Measurement of communication pattern performance

Here we use a communication micro-benchmark that implements the logical shift communication pattern and is parameterized in terms of message size and processor count. The micro-benchmark was implemented using the coNCePTuaL network correctness and performance testing utility [12]. The pseudo code of the benchmark is:

Barrier, Time T1
For n repetitions do
    ISend to self + distance,a message of S bytes
    IRecv from self – distance, a message of S bytes
    Waitall
Barrier, Time T2

where *distance* is the logical shift distance. This benchmark was used to obtain the time taken to

perform the logical shift communication pattern for S = 1Mbytes for the three maps. Figure 3 shows the time taken (in ms on the right-hand Y-axis) for the logical shift communication pattern for a shift distance up to 128. Also shown is the contention using the routing tables presented in Section 3. The scale of the Y-axis was deliberately chosen to examine the correspondence between the measured time and the modeled contention. Exact correspondence can be seen for the typical map, very good correspondence for the optimum case, and exact correspondence up to a shift distance of 48 in the worst-case.

In the case of the optimized map, the contention increases linearly to 4 and then plateaus. The height of this plateau is simply the size of a compute-node in terms of processing cores (assuming a single MPI process per core). A maximum of 4 is reached when all 4 processors perform an inter-node communication (at a shift distance of 4 or greater). Since the optimized map allows all 12 up-links to be simultaneously used to communication from one level-1 switch to another, no further contention occurs (i.e. the optimized map has 1/12 the contention of the worst-case map at shift-distances of 48 or greater).

The use of a typical map results in a contention factor that is between the worst-case and the best-case, and that generally increases with shift-distance. Though the contention level may also decrease as shown in Figure 3 as a result from the precise routing tables employed and the communication pattern (or shift distance in this case) used. In the experience that we have gained from our test-system, the typical case is close to the optimized case – though we expect the typical case to get worse as a system scales in size.

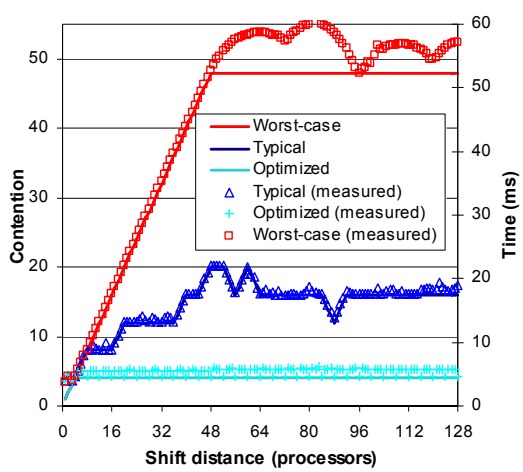


Figure 3. Comparison of modeled contention (left Y-axis) and measured communication time (right Y-axis) for a 1024-processor job

6. Impact on application performance

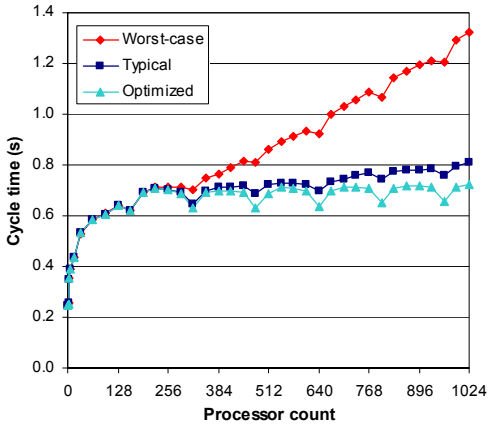
In this section we examine the effect of using different maps on application performance. Two large-scale applications, SAGE and Partisn are used in this analysis. SAGE is an adaptive mesh hydrodynamics application that is used for the simulation of shock-waves. The performance characteristics of SAGE have been studied and modeled in some detail [9]. Partisn is an implementation of S_N transport, the solution of the Boltzmann equation using the discrete ordinates method, on structured meshes. Some details on Partisn can be found in [2]. Both applications are many hundreds of thousands of fortran90, and can run on thousands of processors for long periods of time. They are iterative and many iteration cycles are undertaken in a physical simulation. A summary of the characteristics of both applications as well as the input decks used in this analysis is listed in Table 1.

The input used for SAGE contains a 2-D global data grid which is partitioned in only 1 dimension. A 3-D problem is used in Partisn and is partitioned in two dimensions. Both applications are run in weak scaling mode in which parallelism is used to increase the resolution of the simulation. In weak-scaling mode, the run-time (or time per cycle) is ideally a constant but in general increases with scale due to parallel overheads including communication costs.

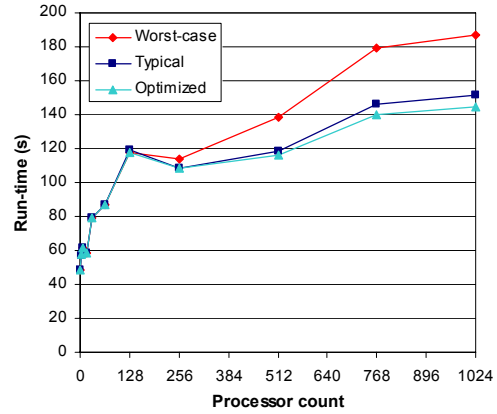
SAGE exhibits a logical shift communication pattern (described earlier). The shift distance gradually increases from 1, at small scale, up to a distance of 24 at 1024 processors for the input deck considered. The 2-D data decomposition used in Partisn results in a shift distance of 1 for communications in the logical X dimension, and a shift distance of P_x in the logical Y dimension. The dimensions of the logical 2-D processors array ($P_x \times P_y$) in Partisn is chosen to be close to a square and thus varies between 1 and 32 when using up to 1024 processors.

Table 1. Application Characteristics

	SAGE	Partisn
Input Deck	Rbeq_5	Rep1
Problem calculation	2-D	3-D
Scaling	Weak	Weak
Sub-grid size (cells/PE)	12,800	~13,500
Logical topology	1-D (+)	2-D mesh
Cycle time on 1 node (s)	0.7	60
Message count /PE/cycle	~4K	~600K
Message sizes (bytes)	8, ~50K	8, ~1K, ~15K
Collectives	Allreduce Alltoall, Reduction	Allreduce, Broadcast



(a) SAGE



(b) Partisn

Figure 4. Application performance using worst-case, typical, and optimized routing tables

The performance of both applications was measured using the typical, worst-case and optimized maps. The measured cycle time for SAGE is shown in Figure 4(a) and for Partisn is shown in Figure 4(b). It can be seen that the highest level of performance (lowest time) is achieved when using the optimized map, worse performance is achieved when using the typical map, and substantially worse performance is achieved when using the worst-case map. The actual improvement in performance, from the typical map to the optimized map, increases with scale and is up to 16% for SAGE and 8% for Partisn. We expect that this difference will be even greater for larger systems due to the bulk synchronous behavior of the applications, if one MPI task is delayed due to contention, all ranks wait for it at the next collective operation.

It should be noted however that the network map optimization only affects a part of the total execution time. In Figure 5 we show the relative performance of just the communication cost of the typical and worst-case maps relative to the optimized map. A value greater than one includes a lower performance, for example a relative performance of 1.2 indicates that the communication cost is 20% longer.

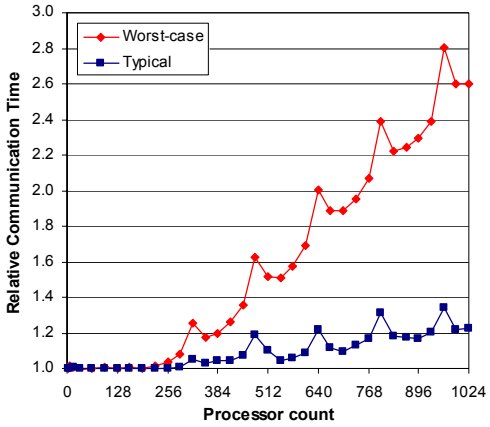
It can be seen in Figure 5 that the communication performance when using the optimized map is up to ~40% faster in SAGE, and 10% faster for Partisn. The difference is substantially greater when considering the relative communication time between the worst-case and the optimized map – up to2.8x on SAGE, and 1.9x on Partisn. SAGE has a communication distance of up to 24 at the largest scale for its major communications whereas Partisn has a distance of up to 32 for only half of its communications (in only one of its two decomposition dimensions) and in addition many of its communications are small. This to a great extent explains the higher improvement observed in SAGE than in Partisn.

7. Conclusions

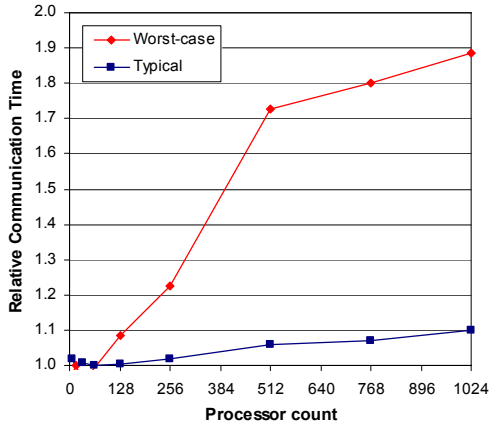
In this work we have quantified the effect of static routing in a large-scale Infiniband cluster for a range of communication patterns. Contention can impact communication performance by up to a factor of 12 higher than in an optimized case. Two custom routing tables for the network were defined: a worst-case map in which only one up-link from a switch is used to route messages, and an optimized map which uses all 12 up-links from a switch simultaneously for a logical shift communication pattern.

Through an analysis of the routing tables for a particular communication pattern it was shown how the maximum contention within the network could be calculated. Further, a set of micro-benchmarks confirmed that this maximum contention did indeed determine the time for a communication pattern. Two large-scale applications of interest to LANL showed that the optimization of the routing tables resulted in improvements in application performance. Overall the run-time on the applications decreased by between 10-20% at the largest scale analyzed (1024 processors on 256 compute-nodes). Further the communication component of the run-time decreased by between 20-40% at the largest scale. We expect the performance improvements to be even greater at larger scale. The optimization of the routing tables presented in this work has applicability to many currently installed Infiniband systems.

From these results further research into adaptive routing for Infiniband is warranted. Barriers to this work include the time it takes to load these routes to switches (several seconds). Possible solutions depend on the persistence of the communication patterns within the applications and also what portion of this work if any could be moved into the HCA or switches.



(a) SAGE



(b) Partisn

Figure 5. Communication time relative to optimum when using worst-case and typical tables

Acknowledgements

This work was funded in part by the Accelerated Strategic Computing program and the Office of Science of the Department of Energy. Los Alamos National Laboratory is operated by Los Alamos National Security LLC for the US Department of Energy under contract DE-AC52-06NA25396.

References

[1] N.R. Adiga, et. al. "An overview of the BlueGene/L Supercomputer", in Proc. of IEEE/ACM Supercomputing, Baltimore, Maryland, 2002.

[2] R.S. Baker, "A Block Adaptive Mesh Refinement Algorithm for the Neutral Particle Transport Equation". Nuclear Science & Engineering, 141(1):1-12, 2002.

[3] K.J. Barker, et. al. "On the Feasibility of Optical Circuit Switching for High Performance Computing Systems", in proc. Supercomputing (SC05), Seattle, WA, 2005.

[4] J. Beecroft, D. Addison, D. Hewson, M. Mclaren, D. Roweth, F. Petrini, and J. Nieplocha, "Quadrics QsNet II: Pushing the Limit of the Design of High-Performance Networks for Supercomputers", Micro, 24(4): 34-47, 2005.

[5] A. Ding, R.R. Hoare, A.K. Jones, R. Melhem, "Level-Wise Scheduling Algorithm for Fat Tree Interconnection Networks", in proc. Supercomputing (SC06), Tampa, 2006.

[6] A. Hoisie, G. Johnson, D.J. Kerbyson, M. Lang, and S. Pakin, "A Performance Comparison Through Benchmarking and Modeling of Three Supercomputers: Blue Gene/L, Read Storm and ASC Purple", in proc. SuperComputing (SC06), Tampa, FL, 2006.

[7] Infiniband Trade Assoc., http://www.infinibandta.org/

[8] S. Kamil, A. Pinar, et. al., "Reconfigurable Hybrid Interconnection for Static and Dynamic Scientific Applications", Tech Report 60060, Lawrence Berkeley National Laboratory, CA, 2006.

[9] D.J. Kerbyson, H.J. Alme, A. Hoisie, F. Petrini, H.J. Wasserman, and M.L. Gittings, "Predictive Performance and Scalability Modeling of a Large-scale Application", in proc. Supercomputing (SC01), Denver, CO, 2001.

[10] D.J. Kerbyson, K.J. Barker, "Automatic Identification of Application Communication Patterns via Templates", in Proc. ISCA Int. Conf. on Parallel and Distributed Computing Systems (PDCS), Las Vegas, NV., 2005.

[11] D.J. Kerbyson, "A Look at Application Performance Sensitivity to the Bandwidth and Latency of Infiniband Networks", in proc. Communication Architectures for Clusters (CAC), Int. Parallel and Distributed Processing Symposium (IPDPS), Rhodes, Greece, 2006.

[12] S. Pakin, "coNCePTuaL: A Network Correctness and Performance Testing Language", in proc. Int. Parallel and Distributed Processing Symposium (IPDPS), Santa Fe, 2004.

[13] E. Papaefstathiou, and D.J. Kerbyson,. "Predicting Communication Delays of Detailed Application Workloads", in proc. Int. Conf. on Parallel and Distributed Computing Systems, (PDCS), Orlando, FL, 2000.

[14] "Pathscale Infinipath HTX Adapter: Low-Latency Cluster Interconnect for Infiniband", available from http://www.pathscale.com

[15] Sandia National Laboratories. Thunderbird Linux Cluster. http://www.cs.sandia.gov/platforms/Thunderbird.html.

[16] Virginia Tech. System-X. http://www.tcf.vt.edu/systemX.html.

[17] E. Zahavi, G. Johnson, D.J. Kerbyson, M. Lang, "Optimized InfiniBand Fat-Tree Routing for Shift All-to-All Communication Patterns", in proc. International Supercomputing Conf., Dresden, Germany, June, 2007